1 UAVCAN 简述：

舵机 CAN 总线协议是基于无人机的一种总线协议,可参考 UAVCAN 协议：

https://uavcan.org/

https://uavcan.org/Examples/Simple_sensor_node/

参考版本：UAVCAN V0

我公司舵机 UAVCAN 最大支持 18 个舵机，舵机编号为 0 到 17。

通讯波特率：1000Kb/s(默认)

建议：舵机 0 设置 Servo ID=0，Local Node ID=100；舵机 1 设置 Servo ID 与 Local Node ID 加 1，其它依次类推。

程序说明如下：

1.1 帧尾（tail byte）:CAN 通讯每帧可传输 1 到 8 个字节，最后一个字节为帧尾；定义如下：

	S	E	T	Transfer				
80	1	0	0	0	0	0	0	0
60	0	1	1	0	0	0	0	0

bit7：开始位（Start bit,简写 S）；=1，表示这条 8 字节数据为开始数据包。

bit6：结束位（End bit,简写 E）；=1，表示这条 8 字节数据为的最后一条数据。

bit5：交替位（Toggle bit,简写 T）；同一个舵机数据包的相邻两条数据为不同值。

bit4-0：数据包编号（Transfer）；值由 0 增大到 31，再回到 0；相邻帧编号相同的是同一包数据。

1.2 校验码（CRC）：适应与数据包大于 1 帧的数据，第一帧数据的前两个字节为效验码。

效验字节数：为收到的数据包全部数据，且不包含 CRC 与 tail byte 的其它字节。

CRC 使用函数：

```
uint16_t crcAddByte(uint16_t crc_val, uint8_t byte){
    crc_val ^= (uint16_t) ((uint16_t) (byte) << 8U);
    for (uint8_t j = 0; j < 8; j++)
    {
        if (crc_val & 0x8000U)
        {
            crc_val = (uint16_t) ((uint16_t) (crc_val << 1U) ^ 0x1021U);
        }
        else
        {
            crc_val = (uint16_t) (crc_val << 1U);
        }
    }
    return crc_val;
}
uint16_t crcAddSignature(uint16_t crc_val, uint64_t data_type_signature){
    for (uint16_t shift_val = 0; shift_val < 64; shift_val = (uint16_t)(shift_val + 8U))
    {
        crc_val = crcAddByte(crc_val, (uint8_t) (data_type_signature >> shift_val));
    }
    return crc_val;
}
uint16_t crcAdd(uint16_t crc_val, const uint8_t* bytes, size_t len){
    while (len--)
    {
    crc_val = crcAddByte(crc_val, *bytes++);
    }
    return crc_val;
}
```

2 舵机反馈工作数据：

反馈频率　　：1HZ、2HZ、5HZ、10HZ（默认）、50HZ；可用编程卡修改

帧 ID 组成　　：

字段	十六进制	十进制	备注
Priority	0x18	24	优先级，不可修改
Data Type ID	0x7E3	2019	反馈工作数据，不可修改
Local Node ID	0x64	100(默认值)	可用编程卡修改

帧 ID = (Priority << 24) | (Data Type ID <<8) | (Local Node ID)

=0x1807E364

示例如下：采集到的数据（蓝色框为 tail byte，红色框为一包数据）

ID号	帧类型	帧格式	长度	数据
0x1807e364	数据帧	扩展帧	8	a8 a0 02 39 72 01 f0 9c
0x1807e364	数据帧	扩展帧	8	81 ed 90 00 9c e4 00 7c
0x1807e364	数据帧	扩展帧	8	0e 2f 02 39 72 01 f2 9d
0x1807e364	数据帧	扩展帧	8	81 ed 90 00 9c e4 00 7d
0x1807e364	数据帧	扩展帧	8	a8 a0 02 39 72 01 f0 9e
0x1807e364	数据帧	扩展帧	8	81 ed 90 00 9c e4 00 7e
0x18015564	数据帧	扩展帧	8	e0 02 00 00 03 30 00 df
0x1807e364	数据帧	扩展帧	8	a8 a0 02 39 72 01 f0 9f
0x1807e364	数据帧	扩展帧	8	81 ed 90 00 9c e4 00 7f
0x1807e364	数据帧	扩展帧	8	a8 a0 02 39 72 01 f0 80
0x1807e364	数据帧	扩展帧	8	81 ed 90 00 9c e4 00 60
0x1807e364	数据帧	扩展帧	8	0e 2f 02 39 72 01 f2 81
0x1807e364	数据帧	扩展帧	8	81 ed 90 00 9c e4 00 61
0x1807e364	数据帧	扩展帧	8	a8 a0 02 39 72 01 f0 82
0x1807e364	数据帧	扩展帧	8	81 ed 90 00 9c e4 00 62
0x1807e364	数据帧	扩展帧	8	0d 03 02 39 72 01 ee 83
0x1807e364	数据帧	扩展帧	8	81 ed 90 00 9c e4 00 63

2.1 接收数据包识别（由 tail byte 的 Transfer 可得到以下两帧是一包数据）：

低位 → 高位

	0	1	2	3	4	5	6	7
A	a8	a0	02	39	72	01	f0	80
B	81	ed	90	00	9c	e4	00	60

2.2 校验码（CRC）：

效验字节，即一包数据不包含 CRC 与 tail byte 的其它字节。

计算示例：

```
uint8_t buffer[12] = { 0x02,0x39,0x72,0x01,0xf0, 0x81,0xed,0x90,0x00,0x9c,0xe4,0x00};
crc = 0xFFFF;
crc = crcAddSignature(crc, 0xCA8F4B8F97D23B57 );
crc = crcAdd(crc, &buffer[0], sizeof(buffer));                    //crc= 0xa0a8
```

2.3 数据字节：

低位 → 高位

A2	A3	A4	A5	A6	B0	B1	B2	B3	B4	B5	B6
02	39	72	01	f0	81	ed	90	00	9c	e4	00

解码如下：

低

高

编号	数据	二进制	低字节	高字节	转数据	工作值	数据类型	功能
0	02	0000 0010	00000		0	0	uint5	Servo_ID
1	39	0011 1001	0100 0111	0010	583	583	uint12	PWM Input
2	72	0111 0010	1110 0100	0000 0011	+996	+9.96°	int16	Pos Cmd
3	01	0000 0001						
4	f0	1111 0000	1110 0001	0000 0011	+993	+9.93°	int16	Pos Sensor
5	81	1000 0001						
6	ed	1110 1101	1101 1011	0010	731	7.31V	uint12	Voltage
7	90	1001 0000	0000 0000	00	0	0A	uint10	Current
8	00	0000 0000	0100 1110	01	334	26.8℃	uint10	PCB Temp
9	9c	1001 1100						
10	e4	1110 0100	1100 1000	00	200	0℃	uint10	Motor Temp
11	00	0000 0000	0 0000		0	0	uint5	StatusInfo

功能说明：

Servo_ID; //Range: 0-17, support up to 18 Servos

PWM_INPUT; //Range:PWM command：700-2300, 1 LSB: us， CAN command：0-1000；

POS_CMD; //Range: -18000-18000,1LSB: 0.01 degree

POS_SENSOR; //Range: -18000-18000, 1 LSB: 0.01 degree

VOLTAGE; //Range: 0-4095, 1 LSB: 0.01 V

CURRENT; //Range: 0-1023, 1 LSB: 0.01 A

PCB_Temp; // Range: 0-1023, 1 LSB: 0.2 degree(Celsius)；temp=dat*0.2-40;

MOTOR_Temp; // Range: 0-1023, 1 LSB: 0.2 degree(Celsius)； temp=dat*0.2-40;

StatusInfo; // uint5,

Bit0：初始化异常报警位；

=0:正常工作,

=1:故障原因:上电没收到有效的命令(收到命令后此原因取消)、

ADC 未校正,、

设置参数异常；

Bit1：工作中控制命令丢失；

=0:正常,

=1:上电收到命令后，然后丢失控制命令(根据命令来源识别)；

注：如果上电没有收到有效命令，此时 BIT0=1，则 BIT1 任然=0，

Bit2：=0:正常, =1:舵臂失力(收到 CAN 命令 1020)；

Bit3：=0:正常, =1:舵机处于堵转状态；

Bit4：=0:正常, =1:高温（任何温度值≥85℃时）；

备注：上面示例中 MOTOR_Temp=0℃，是因为测试舵机没有此功能，在其它舵机中有此功能。

3 舵机接收的控制命令

命令频率　　：大于 200hz（CAN 波特率 1000kb/s、反馈速度 50hz，实测 18 个舵机通讯正常）。

帧 ID 号组成 ：

字段	十六进制	十进制	备注
Priority	0x18	24	优先级，不可修改
Data Type ID	0x07E2	2018	发送控制命令，不可修改
Source Node ID	0x01	1(默认值)	可用编程卡修改

帧 ID = (Priority << 24) | (Data Type ID <<8) | (Source Node ID)

= 0x1807E201

注：发送舵机命令必须从 Servo_ID=0 到最大舵机编号，中间不能有跳跃，

例如：发送舵机 Servo ID=4，则必须发送 5 个舵机的命令（即舵机编号：0、1、2、3、4）；

3.1 示例：5 个舵机控制命令示例（发送命令格式适用于 1 到 5 个舵机）：

ID号	帧类型	帧格式	长度	数据
0x1807e201	数据帧	扩展帧	8	64 32 02 c6 41 f4 40 da
0x1807e201	数据帧	扩展帧	8	64 32 02 c6 41 f4 40 db
0x1807e201	数据帧	扩展帧	8	64 32 02 c6 41 f4 40 dc
0x1807e201	数据帧	扩展帧	8	64 32 02 c6 41 f4 40 dd
0x1807e201	数据帧	扩展帧	8	64 32 02 c6 41 f4 40 de
0x1807e201	数据帧	扩展帧	8	64 32 02 c6 41 f4 40 df
0x1807e201	数据帧	扩展帧	8	64 32 02 c6 41 f4 40 c0
0x1807e201	数据帧	扩展帧	8	64 32 02 c6 41 f4 40 c1
0x1807e201	数据帧	扩展帧	8	64 32 02 c6 41 f4 40 c2
0x1807e201	数据帧	扩展帧	8	64 32 02 c6 41 f4 40 c3
0x1807e201	数据帧	扩展帧	8	64 32 02 c6 41 f4 40 c4
0x1807e201	数据帧	扩展帧	8	64 32 02 c6 41 f4 40 c5
0x1807e201	数据帧	扩展帧	8	64 32 02 c6 41 f4 40 c6
0x1807e201	数据帧	扩展帧	8	64 32 02 c6 41 f4 40 c7
0x1807e201	数据帧	扩展帧	8	64 32 02 c6 41 f4 40 c8

3.1.1 如上图对应数据包数据如下：

低位 → 高位

0	1	2	3	4	5	6	7
64	32	02	c6	41	f4	40	c2

3.1.2 校验码（CRC）：无

说明：当发送舵机命令数小于等于 5 个时，只需 1 帧数据，所以不需要 CRC 效验。

3.1.3 数据字节（不含 CRC 与 tail byte）：

低位 → 高位

A2	A3	A4	A5	A6	B0	B1
64	32	02	c6	41	f4	40

解码如下：

低 ↓ 高

编号	数据	二进制	低字节	高字节	命令值	数据类型	Servo ID
0	64	0110 0100	0110 0100	00	100	uint10	0
1	32	0011 0010	1100 1000	00	200	uint10	1
2	02	0000 0010	0010 1100	01	300	uint10	2
3	c6	1100 0110	1001 0000	01	400	uint10	3
4	41	0100 0001					
5	f4	1111 0100	1111 0100	01	500	uint10	4
6	40	0100 0000	0000 00		0	无效	无效

3.2 示例：18 个舵机控制命令（**发送命令格式适用于 6 到 18 个舵机**）：

ID号	帧类型	帧格式	长度	数据
0x1807e201	数据帧	扩展帧	8	5e c0 64 25 8c 83 e8 9b
0x1807e201	数据帧	扩展帧	8	2c 57 99 07 09 f4 49 3b
0x1807e201	数据帧	扩展帧	8	a5 8a 2a bc bb a2 0d 1b
0x1807e201	数据帧	扩展帧	5	4b 84 ed b0 7b
0x1807e201	数据帧	扩展帧	8	5e c0 64 25 8c 83 e8 9c
0x1807e201	数据帧	扩展帧	8	2c 57 99 07 09 f4 49 3c
0x1807e201	数据帧	扩展帧	8	a5 8a 2a bc bb a2 0d 1c
0x1807e201	数据帧	扩展帧	5	4b 84 ed b0 7c
0x1807e201	数据帧	扩展帧	8	5e c0 64 25 8c 83 e8 9d
0x1807e201	数据帧	扩展帧	8	2c 57 99 07 09 f4 49 3d
0x1807e201	数据帧	扩展帧	8	a5 8a 2a bc bb a2 0d 1d
0x1807e201	数据帧	扩展帧	5	4b 84 ed b0 7d
0x1807e201	数据帧	扩展帧	8	5e c0 64 25 8c 83 e8 9e
0x1807e201	数据帧	扩展帧	8	2c 57 99 07 09 f4 49 3e
0x1807e201	数据帧	扩展帧	8	a5 8a 2a bc bb a2 0d 1e
0x1807e201	数据帧	扩展帧	5	4b 84 ed b0 7e

3.2.1 如上图对应数据包数据如下：

低位 → 高位

	0	1	2	3	4	5	6	7
A	5e	c0	64	25	8c	83	e8	9d
B	2c	57	99	07	09	f4	49	3d
C	a5	8a	2a	bc	bb	a2	0d	1d
D	4b	84	ed	b0	7d			

3.2.2 校验码（CRC）：

效验字节，即一包数据不包含 CRC 与 tail byte 的其它字节。

计算示例：

```
uint8_t buffer[23] = { 0x64,0x25,0x8c,0x83,0xe8,
                       0x2c,0x57,0x99,0x07,0x09,0xf4,0x49,
                       0xa5,0x8a,0x2a,0xbc,0xbb,0xa2,0x0d,
                       0x4b,0x84,0xed,0xb0};
crc = 0xFFFF;
crc = crcAddSignature(crc, 0x5D09E48551CE9194);
crc = crcAdd(crc, &buffer[0], sizeof(buffer));          //crc= 0xc05e
```

3.2.3 数据字节（不含 CRC 与 tail byte），参考 CRC 中 buffer[23]；

解码方法参考 3.1.3 的方法，解码后舵机命令如下：

Servo ID	CAN CMD (uint10)	**Servo ID** （uint10）	CAN CMD
0	100	9	550
1	150	10	600
2	200	11	650
3	250	12	700
4	300	13	750
5	350	14	800
6	400	15	850
7	450	16	900
8	500	17	950